

18 527.363 400.118C527.1 400.216 526.866 400.376 526.676 400.583C526.488 400.789 526.35 401.037
526.274 401.306L521.695 415.326H503.151L498.571 401.306C498.495 401.037 498.357 400.79 498.169
400.584C497.979 400.378 497.745 400.219 497.484 400.119C497.221 400.02 496.941 399.982 496.662
400.011C496.384 400.04 496.116 400.133 495.88 400.283C495.565 400.483 495.32 400.778 495.183
401.124L488.403 418.824L488.331 419C487.355 421.551 487.235 424.35 487.989 426.976C488.743
429.601 490.33 431.911 492.511 433.556L492.536 433.575L492.594 433.62L502.914 441.355L508.034
445.224L511.145 447.575C511.51 447.851 511.955 448 512.411 448C512.869 448 513.314 447.851
513.679 447.575L516.789 445.224L521.909 441.355L532.303 433.575L532.331 433.553C534.513 431.90
536.1 429.6 536.854 426.975C537.609 424.35 537.489 421.551 536.514 419Z" fill="E24329"></path>
<path d="M536.514 419L536.448 418.82C533.141 419.499 530.028 420.899 527.328 422.923L512.431
434.186L521.916 441.355L532.309 433.575L532.339 433.553C534.517 431.908 536.104 429.599 536.858
426.974C537.61 424.349 537.49 421.55 536.514 419Z" fill="FC6D26"></path>
<path d="M502.914 441.355L508.034 445.224L511.145 447.575C511.51 447.851 511.954 448 512.411
448C512.869 448 513.314 447.851 513.679 447.575L516.789 445.224L521.909 441.355L512.425
434.186L502.914 441.355Z" fill="FCA326"></path>
<path d="M497.519 422.923C494.82 420.9 491.706 419.5 488.403 418.824L488.331 419C487.355
421.551 487.235 424.35 487.989 426.976C488.743 429.601 490.33 431.911 492.511 433.556L492.536
433.575L492.594 433.62L502.914 441.355L512.405 434.186L497.519 422.923Z" fill="FC6D26"></path>
<text x="50%" y="255" class="text devsecops">DevSecOps</text><text x="353" y="131" class="text
faded">Plan</text><text x="127" y="131" class="text faded">Code</text><text x="127" y="371"
class="text faded">Build</text><text x="353" y="371" class="text faded">Test</text><text
x="672" y="131" class="text faded">Release</text><text x="898" y="131" class="text
faded">Deploy</text><text x="898" y="371" class="text faded">Operate</text><text x="672"
y="371" class="text faded">Monitor</text><text x="50%" y="13" class="text text2
highlighted">Security</text><text x="50%" y="488" class="text text2
highlighted">Compliance</text></svg></div><div class="stages"><div class="stage"><div
class="circle"><svg id="Layer1" data-name="Layer 1" xmlns="https://www.w3.org/2000/svg"
viewBox="0 0 46 46"><path d="M45,31.6H26.34a3.48,3.48,0,0,0-
6.68,0H6.42V21.2H39.58v6.4a1,1,0,1,0,2,0V20.2a1,1,0,0,0-1-
1H37V14A14,14,0,0,0,9,14v5.2H5.42a1,1,0,0,0-
1,1V31.6H1a1,1,0,0,0,0,2H19.66a3.48,3.48,0,0,0,6.68,0H39.58V44H6.42V37.6a1,1,0,0,0-
2,0V45a1,1,0,0,0,1,1H40.58a1,1,0,0,0,1-1V33.6H45a1,1,0,0,0,0-
2ZM11,14a12,12,0,0,1,24,0v5.2H11ZM23,34.1a1.5,1.5,0,1,1,1.5-
1.5A1.5,1.5,0,0,1,23,34.1Z"></path></svg></div><div class="stagename">Secure</div></div><div
class="stage"><div class="circle"><svg id="Layer1" data-name="Layer 1"
xmlns="https://www.w3.org/2000/svg" viewBox="0 0 46 46"><path d="M21,27.52a1,1,0,0,0,.7-
.29I7.05-7a1,1,0,0,0-1.41-1.41L21,25.11,18.7,22.86a1,1,0,0,0-
1.41,0,1,1,0,0,0,1.41I3,3A1,1,0,0,0,21,27.52Z"></path><path d="M39.9,29.58a1,1,0,0,0-
1.35A24.84,24.84,0,0,1,23.44C14.4,41.22,7.84,34.33,6.26,26.33a1,1,0,0,0-.1,3.52,3.52,0,1,0-
1.92,21.88.88,0,0,0,.28C6,35.52,13.26,43.07,22.71,46a1.07,1.07,0,0,0,.58,0c7.88-2.41,14.29-
8,17.16-
15.08A1,1,0,0,0,39.9,29.58ZM3.37,23a1.5,1.5,0,1,1,1.5,1.5A1.5,1.5,0,0,1,3.37,23Z"></path><path
d="M42.13,19.66v-13a1,1,0,0,0-1.34-.93,14.54,14.54,0,0,1-17-5.26,1,1,0,0,0-
1.62,0,14.54,14.54,0,0,1-17,5.26,1,1,0,0,0-
1.34,93v8.64a1,1,0,0,0,2,0V8A16.46,16.46,0,0,0,23,2.63,16.46,16.46,0,0,0,40.13,8V19.66a3.5,3.5,
0,1,0,2,0Zm-1,4.84a1.5,1.5,0,1,1,1.5-1.5A1.5,1.5,0,0,1,41.13,24.5Z"></path></svg></div><div
class="stagename">Govern</div></div></div></div>

```
<style>
  .diagramwrapper {
```

```
width: 60%;  
display: flex;  
margin-left: auto;  
margin-right: auto;  
}
```

```
.diagramwrapper .box {  
width: calc(100% - 240px);  
padding: 10px;  
display: flex;  
background-color: fff;  
}
```

```
.diagramwrapper .box svg {  
width:100%;  
}
```

```
.diagramwrapper .stages {  
display: flex;  
width: 210px;  
margin-left: 30px;  
flex-wrap: wrap;  
}
```

```
.diagramwrapper .stage {  
display: flex;  
width: 210px;  
margin: auto 0;  
}
```

```
.diagramwrapper .circle {  
width: 80px;  
height: 80px;  
margin-right: 20px;  
border: 999 1px solid;  
border-radius: 50%;  
display: inline-block;  
background-color: fff;  
padding: 0;  
}
```

```
.diagramwrapper .circle svg {  
fill: 524684;  
width: 50px;  
margin: 15px;  
}
```

```
.diagramwrapper .stagenam {  
display: inline-block;  
font-family: Source Sans Pro,sans-serif;
```

```
font-weight: bold;
font-size: 24px;
line-height: 80px;
}
</style>
```

Teams and Handbook Pages

The following teams comprise the sub-department:

- Software Supply Chain Security stage - handbook
 - Anti-abuse group - handbook
 - Authentication group - handbook
 - Authorization group - handbook
 - Compliance group - handbook
- Secure stage - handbook
 - Composition Analysis group - handbook
 - Dynamic Analysis group - handbook
 - Static Analysis group - handbook
 - Secret Detection group - handbook
 - Vulnerability Research group - handbook
 - API Security - handbook
- Security Risk Management
 - Security Policies group - handbook
 - Threat Insights group - handbook

It is important to delineate who the EM and PM DRIs are for every functionality, especially where this may not be obvious. This is documented on a dedicated delineation page.

Product Direction

Product direction can be found on the Sec Section Product Direction handbook page.

Project Setup

Keeping our projects organized is very important for productivity and maintainability.

- To setup a new project we follow the company-wide Engineering guidelines.
- Sec projects should be organized into one of
 - <https://gitlab.com/gitlab-org/secure>
 - <https://gitlab.com/gitlab-org/software-supply-chain-security>
 - <https://gitlab.com/gitlab-org/security-products>

In general, we want to keep as few projects in security-products as necessary.

security-products should only contain :

- Source code for applications that will run as part of a customer install
- Demos
- Historical projects that are difficult to move.

secure and software-supply-chain-security should have projects for:

- End-to-end testing
- Benchmarks / Stats
- Tooling

There may be projects that should belong in secure or software-supply-chain-security but for technical reasons are much easier to have in security-products. In those cases, we can locate the project in security-products if reasonable efforts were made to get the project in secure or software-supply-chain-security but were unsuccessful.

Recommended settings

When creating a new project, all settings should be left to the default options, except for the following which are specific to the secure stage:

1. Add a CODEOWNERS file to the project, for example:

```
shell
[Maintainers]
@gitlab-org/maintainers/container-scanning

^[Reviewers]
@gitlab-org/secure/static-analysis
```

We recommend creating a dedicated group of maintainers for use in the CODEOWNERS file.

1. Disable the project issue tracker.

- Settings -> General -> Visibility, project features, permissions -> Issues
- Disabled

Issues should be created in the groups/gitlab-org issue tracker instead. See step 3. below to configure this.

Using a single, centralized issue tracker over per-project issue trackers has the following advantages:

- It improves the visibility of issues and aligns with our value of transparency.

For example, it's very easy for community members to filter the issues in the groups/gitlab-org tracker to discover GitLab issues seeking wider community contributions.

- It leverages existing tools and infrastructure, such as having triage-ops and other bots executed against issues, without any additional configuration.

- It provides a more consistent experience, since all labels and issue templates will be the same.

- It's easier to write automated scripts, such as using the Security triage automation tool to create/modify vulnerabilities.

- There are some issues that apply to multiple projects. If each project has their own issue tracker, we'd need to figure out which issue tracker should "own" an issue that applies to multiple projects.

Having said that, there are currently some limitations related to using a single, centralized issue tracker, for example resolving threads in new issues doesn't work.

Until this issue has been resolved, we may choose to leave the Issue tracker enabled in the new project.

In these cases, please consider these to avoid abandoned issues:

1. Make the tracker private.
1. Add an issue template with instructions.
1. Ensure there's a triage process in place.

1. Configure a custom issue tracker

- Settings -> Integrations -> Custom issue tracker -> Configure
 - Enable integration
 - Active
 - Project URL
 - <https://gitlab.com/gitlab-org/gitlab/issues>
 - Issue URL
 - <https://gitlab.com/gitlab-org/gitlab/issues/:id>
 - New issue URL
 - <https://gitlab.com/gitlab-org/gitlab/issues/new>

1. Configure the following project features and permissions settings:

- Settings -> General -> Visibility, project features, permissions
 - Project visibility
 - Public
 - Additional options
 - Users can request access
 - Disabled
 - Container Registry
 - Only Project Members
- Settings -> Repository -> Protected branches
 - Allowed to merge
 - Maintainers
 - Allowed to push and merge
 - No one
 - Allowed to force push
 - Disabled
 - Code owner approval
 - Enabled

- Settings -> Repository -> Protected tags
 - Tag
 - v
 - Allowed to create
 - Maintainers
 - GitLab Dev Service - Secure Stage - Analyzers Automation
- Settings -> Merge Requests
 - Squash commits when merging
 - Require
 - Approval settings
 - Prevent approval by author
 - Prevent editing approval rules in merge requests
 - Remove approvals by Code Owners if their files changed
 - Merge request approvals -> Approval rules
 - Approvers
 - All eligible users
 - Target branch
 - All branches
 - Approvals required
 - 1
 - Merge checks
 - All threads must be resolved
 - Pipelines must succeed
 - Merge commit message template

markdown

Merge branch '%{sourcebranch}' into '%{targetbranch}'

%{title}

%{issues}

See merge request %{url}

Merged-by: %{mergedby}

%{approvedby}

%{reviewedby}

%{coauthoredby}

- Default description template for merge requests

markdown

What does this MR do?

<!--

Describe in detail what your merge request does, why it does that, etc.

Please also keep this description up-to-date with any discussion that takes place so that reviewers can understand your intent. This is especially

important if they didn't participate in the discussion.

Make sure to remove this comment when you are done.

-->

What are the relevant issue numbers?

Does this MR meet the acceptance criteria?

- [] Changelog entry added
-] [Documentation created/updated for GitLab EE, if necessary
- [] Documentation created/updated for this project, if necessary
-] Documentation reviewed by technical writer or follow-up review issue [created
-] [Tests added for this feature/bug
- [] Job definition updated, if necessary
 -] [Auto-DevOps template
 -] [Job definition example
 -] [CI Templates
-] Ensure the report version [matches the equivalent schema version
-] Conforms to the [code review guidelines
-] Conforms to the [Go guidelines
- [] Security reports checked/validated by reviewer

/label ~"devops::secure" ~"Category:" ~"group:~" ~"backend"

When configuring projects that are not part of the secure stage, please see the GitLab Projects Baseline Requirements for more details.

Performance Indicators

- Sec Sub-department Performance Indicators
- Error Budgets as Performance Indicators for stage groups

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "SECTIONLABEL"="sec" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "SECTIONLABEL"="sec" >}}
{{< /tableau >}}
```

Slack channels

- sec-section - Sec Section discussions spanning the Software Supply Chain Security and Secure

stages.

- sec-growth-datascience-people-leaders - Engineering people leaders in Sec, Growth, and ModelOps.
- ·sec-growth-datascience-leadership-confidential - Private channel for engineering people leaders in Sec, Growth, and ModelOps.

Calendars

We have two stage level calendars, Secure Stage Calendar and Govern Stage Calendar, where we host cross-group events such as:

1. Monthly retrospective
1. Coffee chats
1. Staff sync

Each group also has a calendar for team-based discussions, such as the our weekly group syncs.

We encourage utilizing our available Google Groups instead of including individuals as attendees when possible. Along with ensuring the event is represented on individual's calendars for visibility, new team members are automatically added to events (as well as removed when someone departs from a team).

Google Groups

Google groups follow the convention [section]-[stage]-[group], separating multi-word names with and are structured as the following:

- sec-section
- sec-software supplychainsecurity
- sec-securityriskmanagement
- sec-applicationsecuritytesting
- sec-securityriskmanagement-securityinsights
- sec-securityriskmanagement-securitypolicies
- sec-securityriskmanagement-securityplatformmanagement
- sec-securityriskmanagement-securityinfrastructure
- sec-applicationsecuritytesting-staticanalysis
- sec-applicationsecuritytesting-secretdetection
- sec-applicationsecuritytesting-dynamicanalysis
- sec-applicationsecuritytesting-compositionanalysis
- sec-software supplychainsecurity-authentication
- sec-software supplychainsecurity-authorization
- sec-software supplychainsecurity-compliance
- sec-software supplychainsecurity-pipeline security
- vulnerability-research

The members of each google group consists of stable counterparts and the correct eng-dev-[stage]-[group] group of engineers. When stable counterparts change, or team members onboard/offboard the appropriate group should be updated by the EM of the respective group.

Staying Informed and Informing Team Members

- Sec Week In Review Google Document - is an asynchronous weekly document of notables things happening in Sec. The document is inspired by the Engineering Week In Review.
- Slack channels ssecure and ssoftware-supply-chain-security are informative since they are all part of Sec Section.

Planning in the Section

In the vast majority of cases, work is scoped to individual groups within the section. However, there are times when the section needs to design and execute solutions as a coordinated Section or risk creating poor and non-cohesive user experiences.

These initiatives will be orchestrated through epics and issues. Initiatives with the following labels are deemed to fall in this category of work.

- ~section::sec and ~group::not owned; or
- all Sec groups

Process for planning section-wide initiatives

At least once per milestone, Senior Engineering Managers in the section will do the following:

- In partnership with Product Management, initiatives 6 months or older will be evaluated to determine if they're still relevant.
- New initiatives will be triaged, checking their requirements for understandability and completeness. Further, the group most impacted will be identified.
 - In situations where most impacted group is not clear, technical leadership via sec-section will be engaged to help discern which group that might be.
- Group most impacted will be declared DRI for that initiative and are expected to:
 - Produce a high-level implementation plan that will scale for the whole problem.
 - Create implementation issues that are broken down by feature category.
 - The original high-level implementation plan will be included, or at least directly linked, in the created issues.
 - Original issue where implementation plan was debated and created will also be linked to the generated issues.
- Distribute implementation issues to the relevant groups.

Generated issues will be worked through normal prioritization processes as they are distributed to individual groups.

Page Performance

Our team monitors LCP (Largest Contentful Paint) to ensure performance is below our target (currently 2500ms).

LCP Dashboard for Secure owned pages

Working with Product Design

To streamline our workflow and ensure efficient collaboration between the Engineering and

Product Design teams, we have established the following guidelines for UX involvement in merge request (MR) reviews:

Merge Request UX Review Requirement:

- A UX review is required only for work that has been explicitly designed by a Product Designer and should be reviewed by that Product Designer.
- MRs that do not involve work explicitly designed by a Product Designer can be labeled as UX Tech Debt and merged without a UX review.

Handling High Priority UX Reviews:

- If a high-priority task arises that requires a UX review but was not planned during the milestone planning process, it should be discussed with the Product Design Manager for Sec.
- To accommodate this unexpected work, another task from the original milestone plan will need to be deprioritized or dropped.

Exceptions:

- These new guidelines do not apply to the Authentication, Authorization, and Pipeline Security groups, which will continue to operate under their current processes.

Working with Customer Support

The Sec engineering teams do not provide support directly to customers. Instead engineers collaborate with our Customer Support Engineers via the process on the Sec Sub-department support project.

Working on security tooling requests

As GitLab grows, the Sec teams continue to build tooling that makes it easier to securely manage users for our largest customers. In doing so, managing team members on GitLab.com is an excellent use case where the features can be internally dogfooded before they are rolled out to our users. The Security and CorpSec teams can add the label security tooling, section::sec and the respective priority priority::1/2/3 to tag an item that will help in such management of GitLab team members and needs to be added to the backlog. The features page is handy in identifying where a particular functionality may belong, such that the correct EM/PM for the group can be tagged in the issue.

The backlog for these issues can viewed at Sec Security Tooling - issue for individual issues. Each month, product and security counterparts will review these requests and ensure that the priority items are scheduled into the roadmap.

How to work with the Quality team

Frontend Responsibilities

1. Being able to identify what code changes would likely break E2E or System level tests and informing Quality.
1. Not to write E2E tests, but to catch potential failures and communicate gaps in coverage

before landing to master.

Identifying potential breakages

1. Look to see if issue you are working on has existing test coverage. These are the tests likely to fail

1. If you are working around code that contains a selector like `data-qa-selector="<name>"`, then there is likely to be an existing E2E test. Tests can be found by searching our E2E tests in Secure.

Communicating changes that may break tests

Ping the DRI for quality assigned to Secure. You can find the person on the team page. If they are unavailable, then sdeveloperexperience on Slack or the triage DRI dependent on severity.

Section Retrospectives

In addition to our group retrospectives, we facilitate an async Sec Section level retrospective each month. The goal of the section wide retrospective is to review topics that bubbled-up from our group/team retrospectives. Additionally, we identify themes that could be discussed synchronously. We use this doc and an issue created with this template to facilitate the section retrospective.

Key Dates

1. The Monday after the monthly release - Group async retrospective issues are generated. Groups should start contributing topics.

1. The week the milestone ends - Groups hold their retrospectives. Team members bubble-up identified topics and follow-up items (outcomes) to the section retrospective document.

1. The week of the release - Section wide retrospective async review shared in the sec-section Slack channel.

DRI Responsibilities

The DRI for Section-wide retrospectives will be the Senior Engineering Manager. The SEM will find a volunteer if it is needed on specific milestones. The following tasks are executed each milestone:

1. Prior to the async section retrospective, review bubble-up topics and identify 2-3 themes to support async discussion topics.

1. Ask everyone through Slack in sec-section to review the section retrospective document and add comments.

1. Share a summary of the async discussions in Slack in sec-section.

1. Follow up with groups on any identified improvements.

1. Promote, promote, promote!

SECTION: engineering/development/sec/delineate-sec.md

Why does this page exist?

In the spirit of establishing a DRI for each set of functionality where this may not be obvious, the purpose of this page is to explicitly define which engineering group has responsibility for which portions of the product and for specific decisions.

Page/Function responsibilities

Page/Function	PM	Primary Group	Example
Secure Partner Onboarding Docs	{{< member-by-gitlab "abellucci" >}}	Sec by their categories	
Security Configuration		Sec by their categories	
Vulnerability ingestion and DB storage	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	
Dependency ingestion and DB storage	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	
Dependency List	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	Example
License Compliance Page	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	
Interact with findings and vulnerabilities	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	
Merge Request Security Widget	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	Example
Merge Request License Compliance Widget	{{< member-by-gitlab "johncrowley" >}}	Secure:Composition Analysis	Example
Pipeline Security Tab	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	Example
Security Dashboards	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	Example
Security Scanner Integration Documentation	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	
Vulnerability Pages	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	Example
Vulnerability Report	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	Example
Policies	{{< member-by-gitlab "g.hickman" >}}	Govern:Security Policies	Example
Advisories ingestion and DB storage	{{< member-by-gitlab "johncrowley" >}}	Secure:Composition Analysis	
External License ingestion and DB storage	{{< member-by-gitlab "johncrowley" >}}	Secure:Composition Analysis	
Vulnerability Match Job	{{< member-by-gitlab "johncrowley" >}}	Secure:Composition Analysis	
License Match Job	{{< member-by-gitlab "johncrowley" >}}	Secure:Composition Analysis	
CVE ID Request - Workflow and automation	{{< member-by-gitlab "sarahwaldner" >}}	Secure:Vulnerability Management	
CVE ID Request - Platform UI	{{< member-by-gitlab "abellucci" >}}	Govern:Threat Insights	
User Invitation Flows	{{< member-by-gitlab "hsutor" >}}	Govern:Authentication	

Technical Boundaries

Ownership of the end-to-end technical solution spans multiple groups. This section clarifies cross-group maintainership of code artifacts between Threat Insights and the remaining groups in the Sec Section.

Ownership means that the responsible group:

- 1. Is responsible for maintenance of the code.
- 1. Receives attribution in their error budget associated with the relevant code.
- 1. Must be consulted before changes are made, and should review them before being merged.
- 1. Receives priority when requesting maintenance and fixes to changes introduced by another group.

Owners by project/function

This is not a comprehensive list, but includes the main projects and areas of functionality under the Sec Section umbrella.

Comparison and tracking logic

The logic to compare and track vulnerabilities is owned by Threat Insights.

Customisation of this logic that is not generic to all report types is owned by the corresponding group(s) (e.g. Static Analysis group would be the maintainer of any improvement to track SAST vulnerabilities).

It is not always possible to promptly identify the group that owns some code. In the absence of an attribution through error budget, CODEOWNERS, code comments or other means, Threat Insights will identify the owner and, if necessary, handover to the appropriate group.

Security Report Schema

The schemas are owned by the backend team in the analyzer groups responsible for the corresponding categories.

Threat Insights owns the base schema and the generic details schema.

For instance, the Static Analysis group is responsible for the SAST category, so its backend team is responsible for the sast report JSON schema.

Regardless of ownership, Threat Insights must review all changes as they are the consumer of reports that conform to the schemas.

For more information on who to request a review from, see the project guidelines.

Vulnerability Management

This includes all items assigned to Threat Insights under Page/Function responsibilities, and also:

- Database Management System objects
- Object-relational mappings
- Integration points
 - APIs
- Security report ingestion framework

AI Vulnerability Explanation and Vulnerability Resolution

These workflows are owned by Threat Insights as part of Vulnerability Pages (see above). This includes integration into the monorepo, display in Vulnerability pages, the merge request interface for Vulnerability Resolution, and integration into Duo Chat for Vulnerability Explanation. Threat Insights collaborates with AI Framework and Duo Chat to support integration into areas these teams own.

Prompts, test data set curation, and verifying quality of responses for Vulnerability Explanation and Vulnerability Resolution are owned by the relevant groups in Secure based on the type of vulnerability. These groups communicate and collaborate with Vulnerability Research, AI Framework for prompt engineering support, and Model Validation for testing at scale, as needed. These features are currently available for SAST only.

While prompts are owned by teams in Secure, prompt engineering is an important part of software development at GitLab, the AI-Powered DevSecOps platform. Teams within Govern - including Threat Insights - are expected to onboard and be able to support prompt engineering for these features, and assist as needed with updating the prompts in the monorepo through follow the sun coverage.

Metrics and usage monitoring within the application are owned by Govern: Threat Insights. Related issue.

SECTION: engineering/development/sec/secure/_index.md

The Application Security Testing engineering sub-department is responsible for the Application Security Testing Stage of the product.

Vision

To provide content and tools to support the best possible assessment at the earliest possible moment.

Following our single application paradigm, we integrate and build scanning tools to supply security and compliance assessment data to the main GitLab application where we develop our vulnerability management system and other features. While it might be technically feasible, we do not aim at building standalone products that provide this data independently from the GitLab application.

For more details about the vision for this area of the product, see the Secure stage page.

Mission

To support the success of GitLab by developing highly usable, high quality tools for customers to build more secure software.

The Application Security Testing team works on GitLab's Secure stage.

Sub-department development people leaders

```
{{< team-by-manager-slug manager="thomaswoodham" team="Manager(.)Application Security Testing" >}}
```

To contact Application Security Testing stage development people leaders, use the following aliases:

GitLab: @gitlab-org/secure/managers

Slack: @sapplicationsecuritytestingmanagers

Team Members

The following people are permanent members of the Application Security Testing Sub-Department:

Composition Analysis

Team Page: Composition Analysis

```
{{< team-by-departments departments="Secure:Composition Analysis BE Team" >}}
```

```
{{< team-by-departments departments="Secure:Composition Analysis FE Team" >}}
```

Static Analysis

Team Page: Static Analysis

```
{{< team-by-departments departments="Application Security Testing:Static Analysis" >}}
```

Dynamic Analysis

Team Page: Dynamic Analysis

```
{{< team-by-manager-role role="Engineering(.)Manager(.)Secure:Dynamic Analysis" team="Engineer" >}}
```

Vulnerability Research

Team Page: Vulnerability Research

```
{{< team-by-departments departments="Secure:Vulnerability Research BE Team" >}}
```

Stable Counterparts

The following members of other functional teams are our stable counterparts:

```
{{< engineering/stable-counterparts role="Secure" other-manager-roles="Engineering Manager(.)Application Security Testing:Static Analysis|Backend Engineering Manager(.)Secure:Dynamic Analysis|Manager, Software Engineering(.)Secure:Composition Analysis|Senior Engineering Manager(.)Secure|Senior Engineering Manager(.)Secure|Director of Engineering(.)Sec|Engineering Manager(.)Dynamic Analysis" >}}
```

Secure Team

The Application Security Testing Team is responsible for the security checks features in the GitLab platform, and maps to the application security testing transversal stage. You can learn more about our approach on the Application Security Testing Vision page.

The features provided by the Application Security Testing Team are mostly present at the pipeline level, and mostly available as container images. This particularity shapes our processes and QA, which differs a bit from the other stages.

Security Products

We still refer to "Security Products" as the tools developed by the Application Security Testing Team. Hence the home of our projects in GitLab: <https://gitlab.com/gitlab-org/security-products/>.

We strive to maintain a consistent User Experience across our Security Products but we do not enforce consistency at the implementation level.

Each group faces its own challenges and is in the best position to make the technical choices it deems are the most suitable to achieve its goals.

While UX inconsistencies are considered as bugs,

we rely on individual teams to make smart decisions about when consistency is important and when divergence makes more sense

- either because the divergence itself creates a better experience or because of velocity considerations.

Domains of Expertise

SAST

SAST (Static Application Security Testing) refers to static code analysis.

GitLab leverages the power of various opensource tools to provide a wide range of checks for many languages and support.

These tools are wrapped inside docker images which ensure we get a standard output from there.

An orchestrator, developed by GitLab, is in charge of running these images, and gathering all the data needed to generate the final report.

DAST

DAST (Dynamic Application Security Testing) is used to hit a live application.

Because some vulnerabilities can only be detected once all the code is actually running, this method complements the static code analysis.

DAST is relying on OWASP Zed Attack Proxy Project, modified by GitLab to enable authentication.

Dependency Scanning

Dependency Scanning is used to detect vulnerabilities introduced by external dependencies in the application.

Because a large portion of the code shipped to production is actually coming from third-party libraries, it's important to monitor them as well.

Dependency Scanning is relying mostly on the Gemnasium engine.

Fuzz Testing

Coverage-guided fuzzing and API fuzzing are used to automatically input data into applications or web apis that has the potential to cause crashes or bugs. Coverage-guided fuzzing relies on open-sourced language-specific fuzzers. API Fuzzing is based on a proprietary GitLab engine.

License Compliance

License Compliance helps with the licenses introduced by third-party libraries in the application.

Licence management relies on the LicenseFinder gem.

Vulnerability Research

The Vulnerability Research team's purpose is to perform research and develop proofs of concepts that increase the capabilities and effectiveness of the Secure stage.

Skills

Because we have a wide range of domains to cover, it requires a lot of different expertises and skills:

Technology skills	Areas of interest
Ruby on Rails	Backend development
Go	SAST, Dependency Scanning, DAST
Python	DAST
SQL (PostgreSQL)	Dependency Scanning / all
Docker	Container Scanning / all
C	API Security

Our team also must have a good sense of security, with at least basic skills in application security.

We provide tools for many different languages (ex: sast, dependency scanning, license compliance). It means our team is able to understand the basics of each of these languages, including their package managers. We maintain tests projects to ensure our features are working release after release for each of them.

Release process

See Versioning and release process.

QA process

See QA Process for more info.

Vulnerability Management process

Automation

We use the security-triage-automation tool in conjunction with scheduled pipelines in the release project to handle the following tasks:

1. Create security issues for FedRAMP vulnerabilities (Container Scanning results only) still detected on the default branch, executed at least once, on the first day of the month to match with FedRAMP compliance report cadence.

Note that we do not yet automatically create security issues for non-FedRAMP vulnerabilities. Please see the Non-FedRAMP vulnerabilities section for more details.

1. Resolve all vulnerabilities (both FedRAMP and non-FedRAMP) no longer detected on the default branch and close their issues, executed every 2 days.

The Vulnmapper tool also provides some automation to vulnerability management like:

1. Adding labels to security issues to further classify the fix availability (fixavailable, fixunavailable, willnotbefixed, etc.).
1. Creating Deviation Request issues for FedRAMP related security issues that should have one.

Note: Our goal is to centralize automation for vulnerability management in the Vulnmapper tool in the nearest future and standardize our processes across the company. However, so far we're following the existing process based on the security-triage-automation tool.

Automation failures

It's possible that our security automation tooling may fail.

If this occurs, and the issue cannot be immediately resolved, open an issue to track the error. Then, announce the failure in sapplcation-security-testing to raise awareness, and follow the manual security triage process outlined below.

<details>

<summary>View manual process fallback when automation fails</summary>

Manually reviewing and resolving vulnerabilities

On a weekly basis: review the vulnerability report to resolve no longer detected ones and close related issues. Note: It is not necessary to investigate vulnerabilities that are no longer detected.

1. Visit Vulnerability Report Dashboards to verify that there are vulnerabilities that can be resolved.
2. Execute the security-triage-automation tool to resolve vulnerabilities and close their issues. This tool must be executed separately for each project that have vulnerabilities to resolve.
3. Verify in Vulnerability Report Dashboards that vulnerabilities have been resolved.

Manually creating security issues for FedRAMP vulnerabilities

Last working day before the 1st of the month, create security issues for FedRAMP vulnerabilities of the CONTAINERSCANNING type, and CRITICAL, HIGH, MEDIUM, LOW, and UNKNOWN severity levels by executing the security-triage-automation tool to process vulnerabilities for a given project (please make sure to adjust CLI options accordingly). This tool must be executed separately for each project.

Manually creating deviation requests for FedRAMP vulnerabilities

Vulnmapper automatically creates Deviation Requests but may fail for various reasons, such as the absence of analysis from NVD.

In cases where automation fails, you must create the Deviation Requests manually before the issues reach SLA.

To do so, use the following procedure.

1. Open a DR issue with the operational requirement template.
 1. Update the Vulnerability Details section with a link to the advisory (RedHat tracker usually), CVE ID, severity, and CVSS score.
 1. Update the Justification Section with:
 - > The OS vendor has published an updated advisory for <CVEID>, indicating that package <PACKAGENAME> has not yet had a fix released for this vulnerability. Until a fix is available for the package, this vulnerability cannot practically be remediated.
 1. Update the Attached Evidence section with:
 - > As this operational requirement represents a dependency on a vendor-published package to address this vulnerability, no additional evidence has been supplied. Please refer to the linked vendor advisory in the above justification.
 1. Link it to the security issue: /relate <issueid>
1. Update the security issue accordingly

text

/label ~"FedRAMP::Vulnerability" ~"FedRAMP::DR Status::Open"

/milestone %Backlog

</details>

Troubleshooting

GITLABACCESSTOKEN has expired. The automation relies on API requests to manage vulnerabilities and issues on various projects. This requires specific permissions and authentication is

achieved with a Private Access Token generated on the service account gl-service-security-triage (credentials available in 1Password). If the token is expired, a new one (with api scope) must be generated by signing in with this account on gitlab.com and then the new value must be configured in the settings of the release project.

FedRAMP vulnerabilities

To ensure compliance, the management of FedRAMP vulnerabilities is handled by automation. Please check the manual process fallback for additional details.

Non-FedRAMP vulnerabilities

We do not yet have the same automation in place for non-FedRAMP vulnerabilities since it represents a too important volume to manage for our teams and some necessary improvements in the vulnmapper tool are required prior to enabling this. In the meantime, we favor a more specialized approach for these vulnerabilities and there is no standardized process across the groups.

Error Monitoring

500 errors on gitlab.com are reported to Sentry. Below are some quick links to pull up Sentry errors related to Application Security Testing.

StoreSecurityReports Worker - <https://sentry.gitlab.net/gitlab/gitlabcom/?query=is%3Aunresolved+StoreSecurityReportsWorker&statsPeriod=14d>

SyncSecurityReportsToReportApprovalRules Worker - <https://sentry.gitlab.net/gitlab/gitlabcom/?query=is%3Aunresolved+SyncSecurityReportsToReportApprovalRulesWorker&statsPeriod=14d>

Vulnerabilities - <https://sentry.gitlab.net/gitlab/gitlabcom/?query=is%3Aunresolved+vulnerabilities&statsPeriod=14d>

On-Demand DAST - <https://sentry.gitlab.net/gitlab/gitlabcom/?query=is%3Aunresolved+Dast&statsPeriod=14d>

Brainstorming sessions

Our team occasionally schedules synchronous brainstorming sessions as a method of deep-diving on a specific topic.

This approach can be useful in breaking down complexity and deriving actionable steps for problems that lack definition.

These are purposefully freeform to allow for creative problem solving.

When possible, time should be reserved for a list of actions to be taken from the open discussion.

Brainstorming Sessions Doc (Internal):

<https://docs.google.com/document/d/179JL5RzbgSIz2XZewbYn79cuX7vUtteTcoLwUUC5o/edit>

Examples of previous brainstorming topics:

Reduce false positives in Security reports

How to manage occurrence uniqueness identification in the common report format? (CompareKey)
One file with syntax errors should not stop SAST and similar kind of jobs from running

Resources

How to triage a QA test pipeline failure
Beginner's guide to writing end-to-end tests
GitLab QA README
GitLab QA Scenarios
E2E info for GitLab developers
Quality training video material

Product Documentation

As the product evolves, it is important to maintain accurate and up to date documentation for our users. If it is not documented, customers may not know a feature exists.

To update the documentation, the following process should be followed:

1. When an issue has been identified as needing documentation, add the ~Documentation label, outline in the description of the issue what documentation is needed, and assign a Backend Engineer and Technical Writer(TW) to the issue (find the appropriate TW by searching the product categories).
 1. If the task is documentation only, apply a ~Px label.
 1. For documentation around features or bugs, a backend engineer should write the documentation and work with the technical writer for editing. If the documentation only needs styling cleanup, clarification, or reorganization, this work should be lead by the Technical Writer with support from a BE as necessary. The availability of a technical writer should in no way hold up work on the documentation.
- Further information on the documentation process.

Async Daily Standups

Since we are a remote company, having daily standup meetings would not make any sense, since we're not all in the same timezone.

That's why we have async daily standups, where everyone can give some insights into what they did yesterday, what they plan to do today, etc.

For that, we rely on the geekbot slack plugin to automate the process.

Standup messages format

Use the "description in backquote + link to issue" format when mentioning issues in your standup report.

Prepend CI status icons to the answer lines for What did you do since yesterday? to denote the current state:

- for successfully accomplished tasks (:cipassing: emoji)
- for tasks that were due on some period of time but were not accomplished (:cifailing: emoji)
- for tasks currently in progress (:cirunning: emoji)
- for paused or postponed tasks (:cipending: emoji)

any other :ci... icon you find applicable

Example:

What did you do since yesterday?

Accomplished the Spotbugs java analyzer compareKey is not unique <https://gitlab.com/gitlab-org/gitlab-ee/issues/10860>

Still working on Allow guests to create an issue from a vulnerability
<https://gitlab.com/gitlab-org/gitlab-ee/issues/7813>

Catch-up on all emails and threads after the vacation

Slack Channels:

As our teams focus on different areas, we have Geekbot configured to broadcast to separate channels in addition to our common one at [ssecure-standup].

- 1. Composition Analysis: gast-composition-analysis-standup
- 1. Dynamic Analysis: gast-dynamic-analysis-standup
- 1. Secret Detection: gast-secret-detection-standup
- 1. Static Analysis: gast-static-analysis-standup

Recorded meetings

Our important meetings are recorded and published on YouTube, in the Application Security Testing Stage playlist.

They give a good overview of the decision process, which is often a discussion with all the stakeholders. As we are a remote company, these video meetings help to synchronize and take decisions faster than commenting on issues. We prefer asynchronous work, but for large features and when the timing is tight, we can detail a lot of specifications. This will make the asynchronous work easier, since we have evaluated all edge cases.

Calendar

We welcome team members to join meetings that are on our shared calendar. The Application Security Testing Calendar is available to all logged in GitLab team members.

Staying informed

GitLab is an extremely active organization which generates a lot of news and activity each week. Everyone in Application Security Testing are encouraged to keep themselves informed as to what is happening in the larger organization. Everyone is also encouraged to contribute to these channels and communication paradigms when you have information to share.

In addition to this, each group in Application Security Testing conducts a weekly synchronous meeting. These meetings are publicized on the Application Security Testing Calendar mentioned above. As always at GitLab, we strive to make meeting attendance optional.

Keeping others informed

In addition to keeping yourself informed, team members are encouraged to keep others informed as well. Application Security Testing groups have adopted a practice of including the following topics as standing agenda items in their weekly meetings, with example topics for each bullet point.

Current status

Work recently achieved against top priorities for that milestone.

Pre-recorded demos are appreciated and encouraged as part of these updates.

Newly discovered scope or dependencies.

Risks

Issues which are blocked or slowed, impacting whether they can be delivered in the desired timeframe.

Help wanted

Issues or topics on which the team or individuals on the team are getting stuck and could use some help.

Praise

Anyone doing a great job and you want to give them kudos?

Any bit of work which has been delivered that's exceptional?

Engineering Managers are responsible for populating this section of weekly group meetings, though everyone can contribute. In addition to helping the group keep itself informed about what's happening each week, the SEM for Application Security Testing will collect this information weekly and broadcast a curated list to the section.

Technical onboarding

New hires should go through these steps and read the corresponding documentation when onboarding in the Application Security Testing Team.

Every new hire will have an assigned onboarding issue that will guide them through the whole process.

Workflow and Refinement

See Application Security Testing Engineering Planning.

Coding standards and style guidelines

The Application Security Testing Team follows the coding standards and style guidelines outlined in the company-wide Contributor and Development Docs, however, please consult the following guidelines which are specific to the Application Security Testing Team:

Application Security Testing Team Go standards and style guidelines

Cross group collaboration

Some components of the architecture that support Application Security Testing features are shared between multiple groups like the common Go library, the Security Report Schemas, the rails parsers, etc.

Modifying these shared pieces might impact other groups so we should rely as much as possible

on approval rules to ensure such changes are reviewed by the relevant teams before being merged.

Impactless two-way door changes could skip the approval process, please use sound judgement and common sense in such situations.

The author of changes should announce broadly the changes made on these components to raise awareness (weekly meeting agenda, slack channel).

Development of new analyzers

For a complete guide about developing a new analyzer please refer to our user documentation

Technical Documentation

As our product evolves, the engineering teams are researching ways to achieve new functionality and improve our architecture.

The outcome of this research can be found in our Technical Documentation section.

Data Sources

We keep a list of data sources in our internal wiki. This includes advisory databases, package license information, and related data.

Retrospectives

The Application Security Testing sub-department conducts retrospectives at the group level.

Each group's EM or delegated DRI is responsible to prepare and schedule the retrospective sync sessions and the async retrospective issues can be found in the corresponding project.

Analytics

The Application Security Testing group reviews analytics to help understand customers and their usage of the tools. This data helps drive product and technical decisions. The following links show usage of Application Security Testing functionality.

Secure Scan Metrics Dashboard

Cross-functional Backlog

We also track our backlog of issues, including past due security and infradev issues, and total open SUS-impacting issues and bugs.

```
{{< tableau height="600px" toolbar="hidden" src="https://10az.online.tableau.com//site/gitlab/v  
iews/TopEngineeringMetrics16989570521080/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "STAGELABEL"="application security testing" >}}  
{{< /tableau >}}
```


Merged Merge Request Types

MR Type labels help us report what we're working on to industry analysts in a way that's consistent across the engineering department. The dashboard below shows the trend of MR Types over time and a list of merged MRs.

```
{{< tableau height="600px"
src="https://10az.online.tableau.com//site/gitlab/views/MRMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "STAGELABEL"="application security testing" >}}
{{< /tableau >}}
```

Common Links

[application-security-testing in Slack](#)
[Security Glossary](#)

SECTION: engineering/development/sec/secure/planning.md

Overview

Our stage follows the product development flow process, including the workflow labels. This page documents tweaks and additions to the general GitLab Process. If there's a conflict, the stage documentation should take precedence.

Some groups prefer to split the Plan phase into two adjacent steps: Planning breakdown and Refinement. Either way, once planning is complete, issues and epics are ready for scheduling.

Planning breakdown

Epics and issues are selected according to your team's prioritization process, and must have the ~workflow::planning breakdown label applied.

The main questions to be answered are:

1. Are requirements clear enough to understand intent of request?
1. Do we know the boundaries of work to be accomplished? (e.g. code maintained by another team)

If either answer is ·No·, discussion continues with the PM to improve the DRI's understanding of the request.

If both answers are ·Yes·, the DRI estimates whether or not the issue can be delivered in a single milestone. For an epic, it's assumed that it can't be delivered in a single milestone.

When it's determined that an issue cannot be delivered within a single milestone, the DRI works to break it down into multiple issues, each of which can be delivered in a milestone. As much as possible, the issues are independent ·slices· of value (results for customers) so, for example, no mocked UIs or backend work that is inaccessible.

Engineering output:

- Identify and resolve outstanding questions or discussions.
- Notify other teams if the issue is relevant to them in some way.
- Explain any dependencies on other teams in the description, and engage your EM/PM to facilitate early collaboration.
- For epics: create implementation issues within the epic(s).
- Apply the ~workflow::refinement label to all issues.

Refinement

Engineers assigned to refine issues are encouraged to ask questions and push back if issues lack the information required for successful refinement and execution.

Engineering output:

- Ensure issue is ready to be worked on; apply the ~workflow::ready for dev label.
- Confirm that the issue has the correct labels, especially the work type classification label.
- Remove assignee(s) from issue.

Scheduling

The EM is responsible for scheduling work, according to feature, maintenance, and bug prioritization for their team.

Engineering output:

- Apply the correct milestone.
- Apply the ~Deliverable label to issues that the team has committed to delivering in the milestone.

Refinement guidelines

1. Assign yourself the issue. Note the differences for bugs and spikes.
1. Consider using the Implementation issue template as it has the structure used in these guidelines.
1. Ensure the appropriate ~backend or ~frontend label is applied or removed.
1. Check the issue for completeness:
 - It has the necessary designs.
 - The functionality clearly articulated, and there is a decision on how it should function.
 - The technical details are outlined, and discussions are resolved.
 - Dependencies have been called-out.
 - Determine if a feature flag is needed.
1. If the issue is not complete:
 - Tag the relevant people that can help complete the issue and outline what is needed. Tag the EM and PM, so they are aware of the blocker.
1. Ensure the issue is fully understood.
 - Update the issue description with the final description of what will be implemented.
 - Update the issue description with an implementation plan.
 - Optionally, as per your team's customs, add verification steps.

- Ensure the issue title is accurate for the work being done.
 - Open up new issues for 'follow-up' work, or work that was forced out of scope.
 - Assign a work type classification label; e.g. /label ~"feature::enhancement".
1. If your team uses weights, assign one accordingly (except for bugs and spikes).
 1. For complex issues, request a review from another engineer. Teams may have set a weight threshold to require reviews.
 1. Add the ~workflow::ready for dev label and unassign yourself.

When you are done refining, anyone should be able to read the issue description and should know what the issue is solving, how it is solving the problem, and the technical plan for implementing the issue.

In order for someone to understand the issue and its implementation, they should not have to read through all the comments. The necessary information must be in the description, as the single source of truth.

Implementation Plan

A list of the steps and the parts of the code that will need to get updated to implement this feature. The implementation plan should also call-out any responsibilities for other team members or teams.

The goal of the implementation plan is to spur critical analysis of the issue and have the DRI think through what parts of the application will get touched. The implementation plan will also permit other engineers to review the issue and call out any areas of the application that might have dependencies or been overlooked.

The implementation plan might be limited to a single step but it should not be skipped, even when the implementation is simple.

This improves consistency across issues, and communicates that issues have been properly refined.

Verification Steps

Explain how your changes can be verified by another person. This expedites the merge request review process, since reviewers won't need to ask for the information.

It also improves the (~workflow::verification) step by enabling someone else to do the verification, which should reduce the chance of the issue being reopened.

Bug Diagnosis

Note the following differences when refining bugs:

1. As a guideline, spend no more than 1 hour per issue. Bugs that take too long to refine are indicative of a more complex issue.
1. Do not add weight. Our velocity represents the capacity to deliver new, bug-free features.
1. When you hit the time limit for refinement, it's ok to have uncertainty in the Implementation Plan. It's sufficient just to direct where you expect the code change to be or

further steps for diagnosis.

Refinement for Spikes

1. Do not add weights^[^1].

1. Time-box how much time to spend on the issue, and note it in the description; e.g. Time-box: 2d.

1. The deliverable is typically an answer or solution to be used in upcoming issues, along with follow-up issues.

[^1]: a spike doesn't directly add value to users so it shouldn't contribute to our velocity.

The information delivered by a spike is what will be useful to deliver direct value to users.

Refinement for Security Issues

The Security Developer process can be daunting for first-timers. As part of refinement, ask for a volunteer to act as a "Security Issue Release Buddy".

SECTION: engineering/development/sec/secure/qa_process.md

Everything starts with a Merge Request

We expect and require all contributions to our products to go a merge request with a formal review. As such, we follow the Merge Request workflow and code review guidelines articulated in GitLab's developer documentation. We would, however, like to highlight a few items from these documents and add a few additional considerations for reviewers and authors.

Additional considerations for Merge Request reviewers

The best way to unblock a peer or community member is to provide feedback in a timely manner. If you are at capacity and cannot facilitate a review in the SLO to which we aspire, please let folks know in the merge request so another reviewer may be found.

Additional considerations for Merge Request authors

Being a globally distributed organization can, and frequently does, add latency to back-and-forth communication between folks. Don't take it personally if it's taking longer than you expected to get feedback on your changes.

Secure QA Process

The secure analyzers verify merge requests by running a new commit against downstream test projects for their supported languages/frameworks (i.e. the gemnasium analyzer of Dependency Scanning will trigger tests against php, go, and several other test projects). The verification is done by comparing the generated report output against an expected report committed to the analyzer's repository. If analyzer behavior has changed, then the pipeline will fail because the contents of the expected and generated reports will no longer match.

The expectations for the Dependency Scanning, SAST, and Secrets Detection analyzers are stored in the project's qa/expect directory with a sub-directory for each particular framework/language type.

Test Projects

The test projects reside in the Secure tests subgroup and their use and best practices are found in the common README.

Changing Analyzer Behavior

If a change to an analyzer changes the report output, then the expected report must change as well.

In SAST, Dependency Scanning, Secret Detection the expected reports are stored with the analyzer in a path corresponding to the test project in a subdirectory for the language/framework. For example, the expected report for scanning the dependencies of java-maven is stored in gemnasium-maven/qa/expect. This way if an analyzer's behavior changes, the expectation can be changed and packaged in the same commit.

For License Compliance, expectations are stored in the test project's qa/expect directory (e.g. ruby-bundler/qa/expect).

More information about can be found in the Security Products test projects repository.

Testing on OpenShift

We currently do not have automated tests for OpenShift. If you want to see how a change affects the analyzer behavior on OpenShift, you can test it by setting up an OpenShift environment. There are two methods to do this:

1. Automated Script

The distribution team has shared an automated script for setting up an OpenShift cluster. While untested by our group, this method could potentially streamline the cluster creation process.

2. Manual Setup

You can setup an OpenShift cluster following the steps below assuming that you already have access to the GitLab Sandbox Cloud.

1. Login to <https://gitlabsandbox.cloud/>.

1. Create an AWS sandbox account and login to the AWS console by following steps 1 to 3 from Static Analysis Group EC2 development machine setup guide.

1. Once you are logged into the AWS console, you can search for "OpenShift" in the search field in the upper left corner, which should show Red Hat OpenShift Service on AWS under the Services category. ROSA provides an integrated OpenShift experience with AWS. Click on this search result to proceed.

1. You are redirected to the Red Hat OpenShift Service on AWS (ROSA) landing page. Click on the Get started link in the left sidebar, or the Get started orange button.

1. You will then be redirected to the Verify ROSA prerequisites page:

Click on the Enable ROSA HCP and ROSA classic button in the ROSA enablement section, and keep the box I agree to share my AWS account number... checked. The ROSA enablement section will then explain Your request to enable ROSA is pending and may take several minutes to resolve. We recommend keeping this page open so that you can review any errors.

1. After a few minutes, the ROSA enablement section will be updated and display the following messages:

You have agreed to share your AWS account number and email address with red hat.
You have enabled ROSA and HCP and ROSA classic.

1. Click on the Continue to Red Hat orange button in the bottom right corner. This will redirect you to <https://sso.redhat.com> and request you to Log in to your Red Hat account:

1. Click on Log in with Google and use your gitlab.com gmail account. You will be redirected to the Register for a Red Hat account page:

Complete the required fields, using your gitlab.com gmail account for the Email address, then click on Create my account.

1. You should now be redirected to a page to Complete your account connection. Check the I have read and agreed to the terms and conditions checkbox, then click on the Connect accounts button:

1. You'll then be redirected to another page explaining We need a little more information. Complete the required fields, using your gitlab.com gmail account for the Email address, and select Personal for Account type:

1. The Terms and conditions box will now be displayed, click on View Terms and Conditions and accept them:

1. You'll now be redirected to the Red Hat Hybrid Cloud Console Overview screen, and can set up an OpenShift cluster by clicking the Create cluster in the Red Hat OpenShift Service on AWS (ROSA) dialog box:

If you are only planning to use the cluster for testing, it is recommended to select the OSD Trial with a free trial period of 2 months. After this period, the cluster will be deleted. The free trial enables you to upgrade to a paid tier within time frame of two months.

1. Once the cluster is set up, we have to create a user to login to the OpenShift cluster; you can select Cluster List > <your cluster> and select htpasswd for Identity Provider. Under Cluster Roles and Access you can create the user for accessing the OpenShift cluster. Make sure that this user belongs to the groups dedicated-admin and cluster-admins.

1. Afterwards you can click on the Open console button and login with the cluster admin user you created in the previous step.

1. Once you are logged into the OpenShift cluster, you can now install the GitLab Runner Operator by selecting Operators > Operator Hub. You can search GitLab Runner, click on the search result and then click on the Install button as explained here.

1. In order to set up the runner in the OpenShift cluster, you can follow the instructions that are included in the README of the operator.

Note that the runner token referenced in the gitlab-runner-operator README.md can be obtained by selecting CI/CD Settings > Runners > New Runner from a GitLab project.

The runner setup requires the command line tool oc (OpenShift client) to be installed. Before using oc you have to authenticate against your cluster. You can find the authentication credentials by clicking on your OpenShift cluster admin user name and then selecting Copy login comand. Note, that the default runner setup allows containers to be executed with root privileges. In order to restrict access to non root users, you can provide the configuration below:

```
toml
[[runners]]
name = "gitlab-runner"
url = "https://gitlab.com"
executor = "kubernetes"
[runners.kubernetes]
[runners.kubernetes.podsecuritycontext]
runasnonroot = true
runasuser = 1000
```

After applying this configuration with oc create configmap